EWB
COURSES

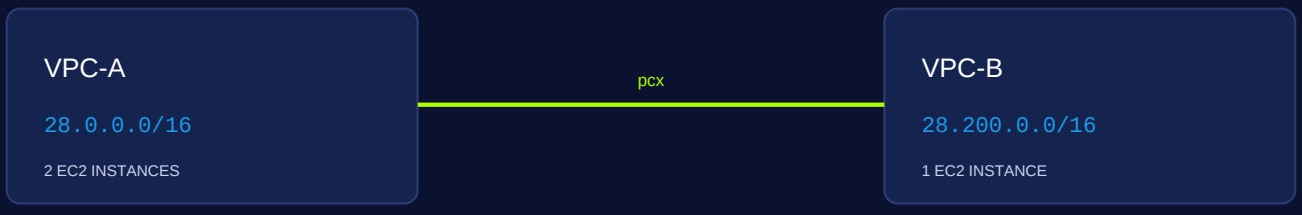
BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.

AWS VPC Peering Between Two Amazon VPCs



2 VPCs / 3 EC2 INSTANCES / 1 PRIVATE CONNECTION

AWS ARCHITECT ASSOCIATE

NAME	**Kancharla Murali Krishna**
BATCH	**Batch 1**
PROJECT TITLE	**AWS VPC Peering Between Two Amazon VPCs**
AWS REGION	**US East (Ohio), us-east-2**
COMPLETION DATE	**22 October 2026**

01 / OVERVIEW

Project objective

Connect two VPCs and check that private communication works.

In this project I connected two Amazon VPCs in the US East (Ohio) Region (us-east-2) using a VPC peering connection, so that an EC2 instance in each VPC can talk to the other using the IPv4 address assigned to it inside its VPC. I set up the route tables and security groups for both sides and tested the connection from the command line with ping and SSH.

What VPC peering is

VPC peering is a private, one-to-one network link between two VPCs. Traffic between them stays on the AWS network and does not need an internet gateway. Creating the peering connection is not enough on its own. Each VPC also needs a route to the other VPC's CIDR block that points to the peering connection, and the security groups must allow the traffic. Peering is not transitive, and the two VPCs cannot have overlapping CIDR blocks.

What I did

- Created two VPCs with CIDR blocks that do not overlap: 28.0.0.0/16 (VPC-A) and 28.200.0.0/16 (VPC-B).
- Created the peering connection VPC-A-to-VPC-B (pcx-00536c471c42e9d73), which shows Active.
- Created Public-subnet-A, Private-subnet-A and Private-subnet-B, each with its own custom route table, and an internet gateway for VPC-A.
- Set up three security groups: Bastion-SG, Appserver-SG and Remoteapp-SG.
- Launched three instances: Bastion-vpc-A, App-server-vpc-A and Remote-app-vpc-B.
- Logged in by SSH to the bastion from my laptop, then from the bastion to the app server in VPC-A, and tried a ping to the instance in VPC-B.
- Saw that ping fail. Section 9 explains what my screenshots can and cannot show about that.

About my CIDR blocks
I used 28.0.0.0/16 and 28.200.0.0/16. These are not RFC 1918 private IPv4 ranges (10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16); they are ordinary public internet address space. The AWS console still labels the instance addresses 'Private IPv4'. A real network should use RFC 1918 ranges instead. The workbook uses 10.10.0.0/16 and 10.20.0.0/16.

About the evidence
Every value in this report comes from my screenshots (Figures 2 to 13, taken on 22 October 2026). If a value is not visible in a screenshot, I have written that. Things the workbook asks for that I did not capture are marked 'Not captured'. Appendix B lists all of them.

02 / ARCHITECTURE

The network I built

Two VPCs, three EC2 instances and one peering connection.

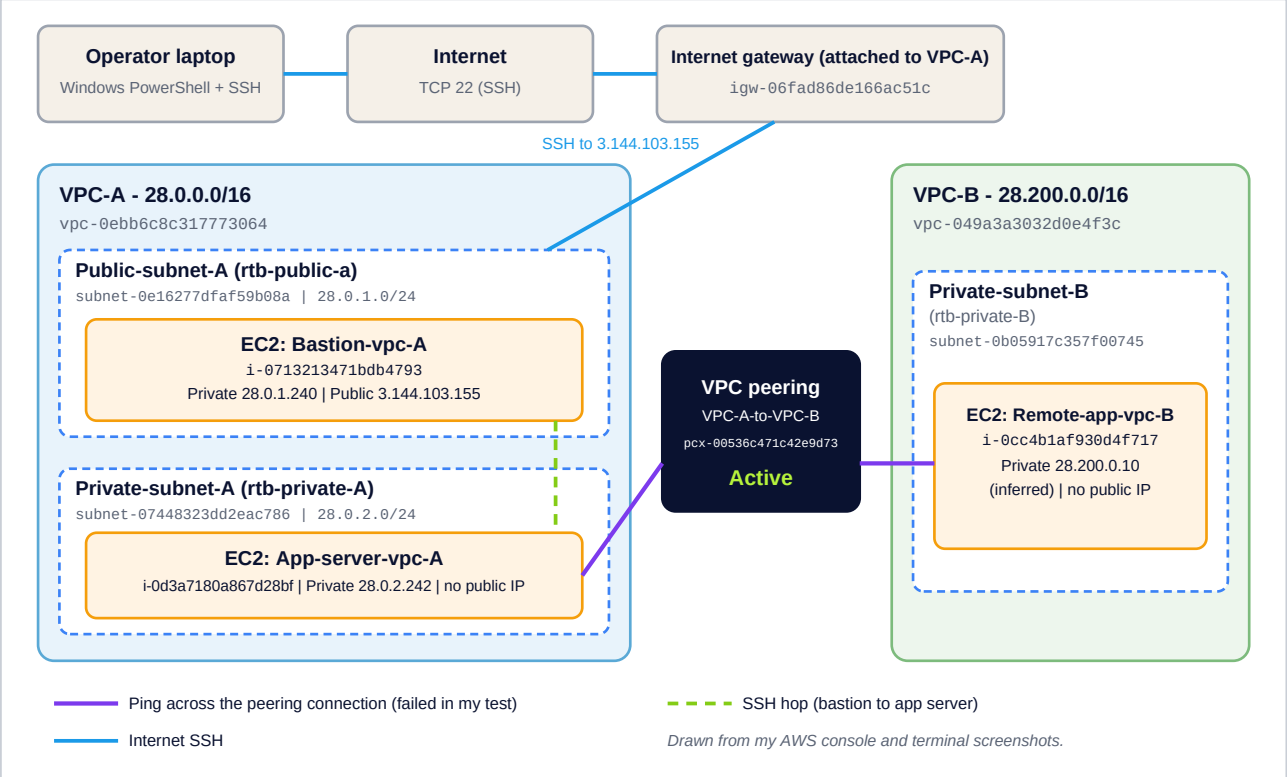


Figure 1. Architecture diagram drawn from my screenshots and notes (not a console screenshot).

Differences from the workbook design

The workbook expects a separate bastion host with a public IP and two application hosts with no public IP, plus three custom route tables and a private subnet in VPC A. My setup follows that design closely. The differences are:

Item	Workbook design	What I built
VPC CIDRs	10.10.0.0/16 and 10.20.0.0/16	28.0.0.0/16 and 28.200.0.0/16 (no overlap, not RFC 1918)
EC2 instances	3: bastion, App-A, App-B	3: Bastion-vpc-A (public IP 3.144.103.155), App-server-vpc-A and Remote-app-vpc-B (no public IPv4)
Bastion	Separate bastion in the VPC A public subnet	Bastion-vpc-A, private IP 28.0.1.240, with Bastion-SG
Private subnet in VPC A	ewb-a-private with its own route table	Private-subnet-A (28.0.2.0/24) with rtb-private-A
Route tables	3 custom tables with explicit associations	3 custom tables with explicit associations: rtb-public-a, rtb-private-A, rtb-private-B
Instance type / OS	t3.micro suggested, Amazon Linux 2023	t3.micro, Amazon Linux 2023 (shown in the terminal banner)
SSH method	ProxyJump with one key pair	ssh to the bastion, key copied to the bastion with scp, then ssh to the app server

Item	Workbook design	What I built
ICMP rule on destination	Echo request from a single /32	All traffic from 28.0.0.0/16 (Remoteapp-SG)

03 / RESOURCE INVENTORY

Resources and IDs

Names, IDs and IP addresses used in this project.

Resource	Name	ID / value
VPC 1	VPC-A	vpc-0ebb6c8c317773064 | 28.0.0.0/16
VPC 2	VPC-B	vpc-049a3a3032d0e4f3c | 28.200.0.0/16
VPC-A public subnet	Public-subnet-A	subnet-0e16277dfaf59b08a | 28.0.1.0/24
VPC-A private subnet	Private-subnet-A	subnet-07448323dd2eac786 | 28.0.2.0/24
VPC-B subnet	Private-subnet-B	subnet-0b05917c357f00745 | 28.200.0.0/16
Internet gateway	IGW-VPC-A	igw-06fad86de166ac51c | attached to VPC-A
Public route table	rtb-public-a	rtb-0fd0f64f7e945d910
VPC-A private route table	rtb-private-A	rtb-00b5393eeb4bb44cc
VPC-B private route table	rtb-private-B	rtb-0759ef2502c2e1bf7
Peering connection	VPC-A-to-VPC-B	pcx-00536c471c42e9d73 | Active
Security group, bastion	Bastion-SG	sg-0656999fdb60ae672
Security group, VPC-A app	Appserver-SG	sg-05e04db63f87f28db
Security group, VPC-B	Remoteapp-SG	sg-0f497072fbef07095
EC2 instance, bastion	Bastion-vpc-A	i-0713213471bdb4793 | private 28.0.1.240 | public 3.144.103.155
EC2 instance, VPC-A app	App-server-vpc-A (*)	i-0d3a7180a867d28bf | private 28.0.2.242 | no public IPv4
EC2 instance, VPC-B app	Remote-app-vpc-B (*)	i-0cc4b1af930d4f717 | private 28.200.0.10 (inferred) | no public IPv4

Notes on the inventory

(*) The Instances list truncates these two names (App-server-vp... and Remote-app-v...). The private IPs of the bastion (28.0.1.240) and App-server-vpc-A (28.0.2.242) come from the terminal prompts in Figures 12 and 13. 28.200.0.10 is the address I pinged, so it is inferred rather than read from the console. The account ID is 841291679285. All three instances are in us-east-2a.

The list also shows a third VPC (vpc-08e14ebf23b0da827) and a second internet gateway (igw-041559c72b1aebc72). They belong to the account's default VPC and are not part of this project.

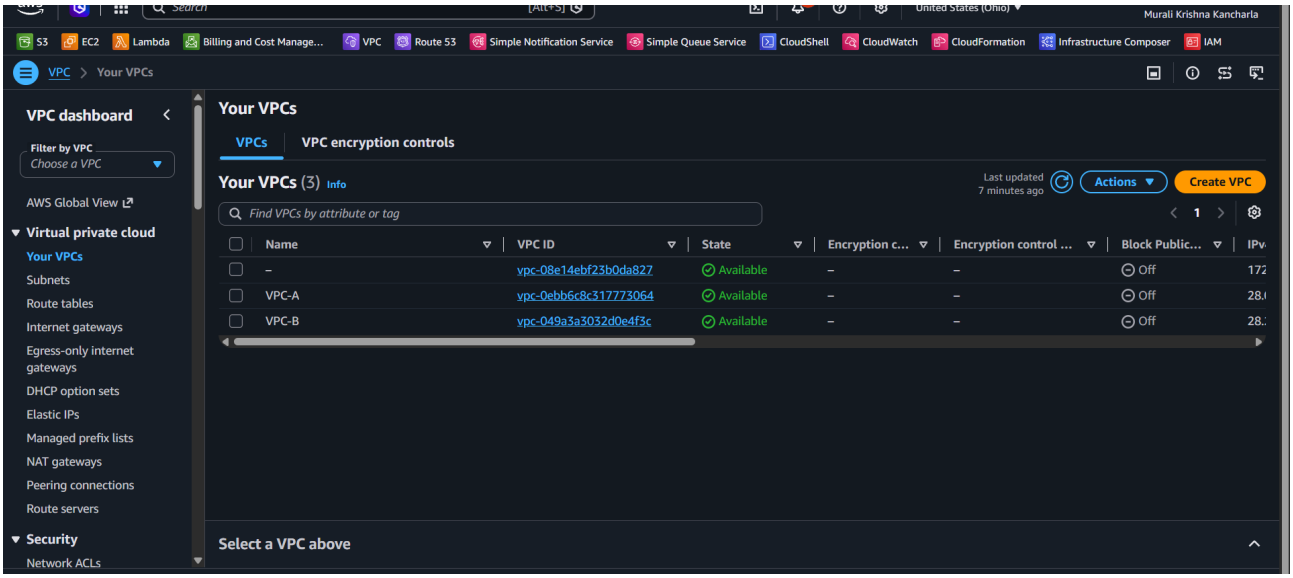


Figure 2. Your VPCs list: default VPC (vpc-08e14ebf23b0da827), VPC-A (vpc-0ebb6c8c317773064) and VPC-B (vpc-049a3a3032d0e4f3c), all Available.

04 / SUBNETS AND ROUTING

Subnets and route tables

Each subnet has its own route table.

Subnets

Subnet	ID	IPv4 CIDR	Block public access	Route table
Public-subnet-A (VPC-A)	subnet-0e16277dfaf59b08a	28.0.1.0/24	Off	rtb-public-a
Private-subnet-A (VPC-A)	subnet-07448323dd2eac786	28.0.2.0/24	Off	rtb-private-A
Private-subnet-B (VPC-B)	subnet-0b05917c357f00745	28.200.0.0/16	Off	rtb-private-B

All three subnets are Available. Private-subnet-B uses the same CIDR as its whole VPC (28.200.0.0/16), as shown in the subnet list. The IP address of Bastion-vpc-A, 28.0.1.240, is inside the public subnet's range, and App-server-vpc-A (28.0.2.242) is inside the private subnet's range.

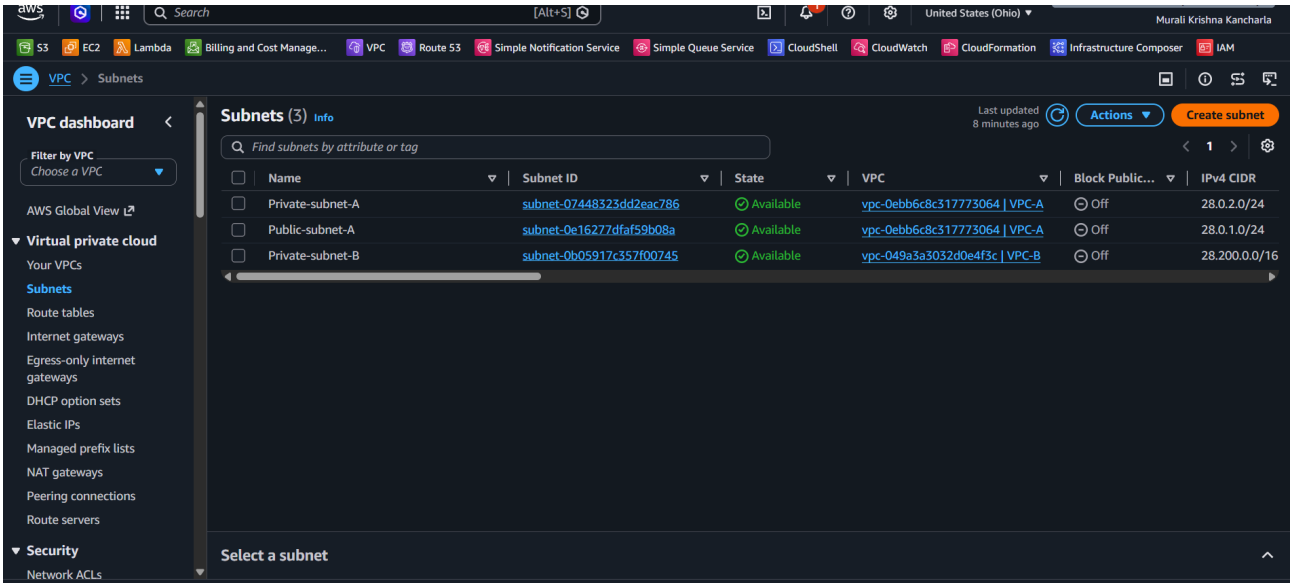


Figure 3. Subnets (3): Private-subnet-A (28.0.2.0/24), Public-subnet-A (28.0.1.0/24) and Private-subnet-B (28.200.0.0/16).

Route tables

The console lists six route tables. Three are my custom tables, each explicitly associated with one subnet. The other three are the main route tables of the default VPC, VPC-A and VPC-B.

Name	Route table ID	Subnet association	Main	VPC
rtb-public-a	rtb-0fd0f64f7e945d910	subnet-0e16277dfaf59b...	No	VPC-A
rtb-private-A	rtb-00b5393eeb4bb44cc	subnet-07448323dd2eac...	No	VPC-A
rtb-private-B	rtb-0759ef2502c2e1bf7	subnet-0b05917c357f00...	No	VPC-B
(unnamed)	rtb-048d74481298fe795	None	Yes	VPC-A
(unnamed)	rtb-0fe6fac7597225422	None	Yes	VPC-B
(unnamed)	rtb-0d3aa779ccfe6662b	None	Yes	Default VPC

The console banner confirms that subnet associations for rtb-private-B were updated. The route entries inside each table (for example 0.0.0.0/0 to the internet gateway, or 28.200.0.0/16 to the peering connection) are not visible in my screenshots, so I have not listed them. I also did not capture a before-peering view of any table.

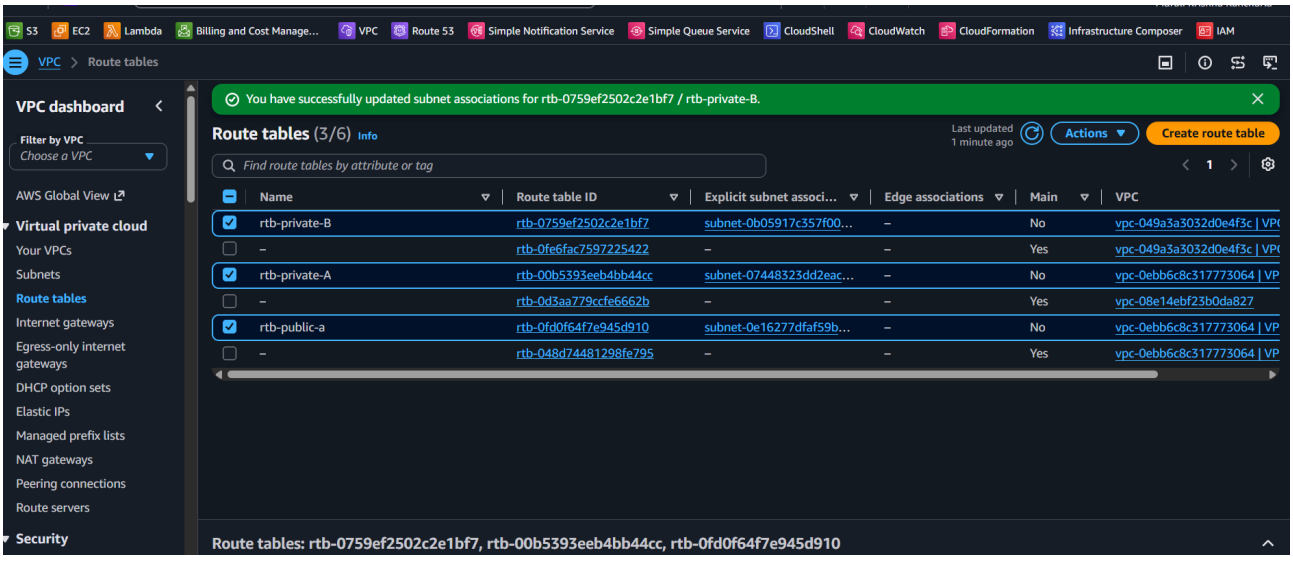


Figure 4. Route tables (3 of 6 selected): rtb-private-B, rtb-private-A and rtb-public-a, each with one explicit subnet association.

Internet gateway

IGW-VPC-A (igw-06fad86de166ac51c) is Attached to VPC-A. The other gateway, igw-041559c72b1aebc72, is attached to the default VPC.

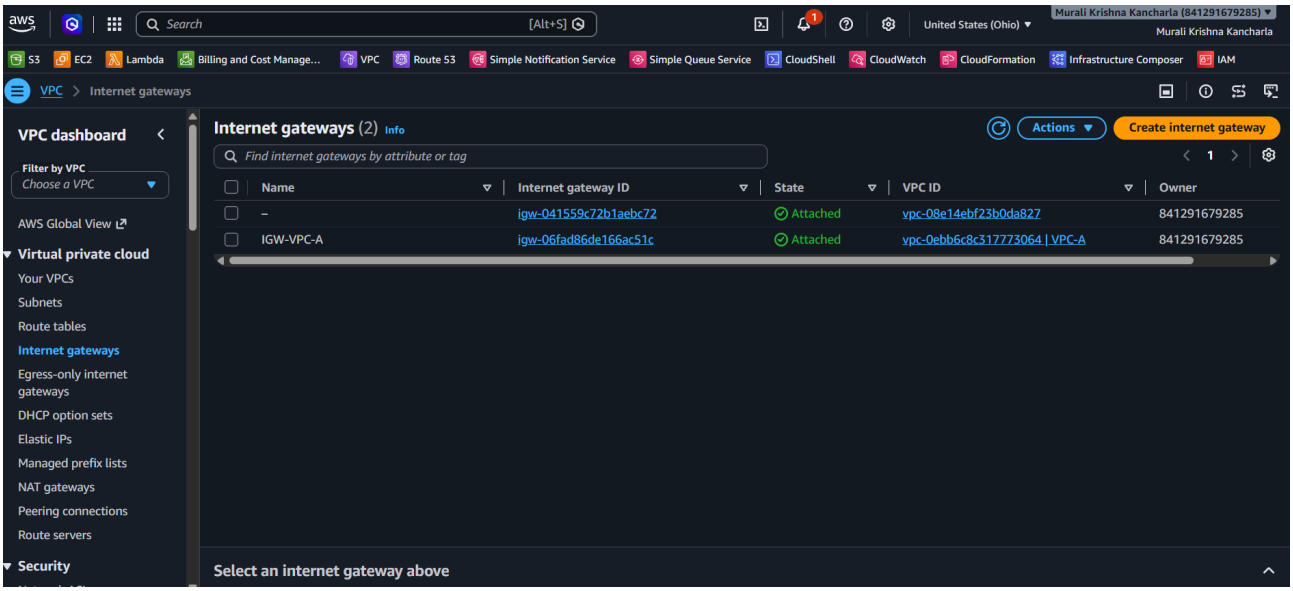


Figure 5. Internet gateways (2): igw-041559c72b1aebc72 (default VPC) and IGW-VPC-A (igw-06fad86de166ac51c, VPC-A), both Attached.

05 / PEERING CONNECTION

Request, accept, activate

The connection is the link. Routes and security groups decide what can use it.

Field	Value (from Figure 7)
Name / ID	VPC-A-to-VPC-B / pcx-00536c471c42e9d73
Status	Active (expiration time is blank)
Requester VPC / CIDR	vpc-0ebb6c8c317773064 (VPC-A) / 28.0.0.0/16
Accepter VPC / CIDR	vpc-049a3a3032d0e4f3c (VPC-B) / 28.200.0.0/16
Requester and accepter Region	Ohio (us-east-2) for both
Requester and accepter owner	Same AWS account (both owner IDs are 841291679285)
DNS resolution over peering	Disabled in both directions. My tests used IP addresses, so I did not need it.

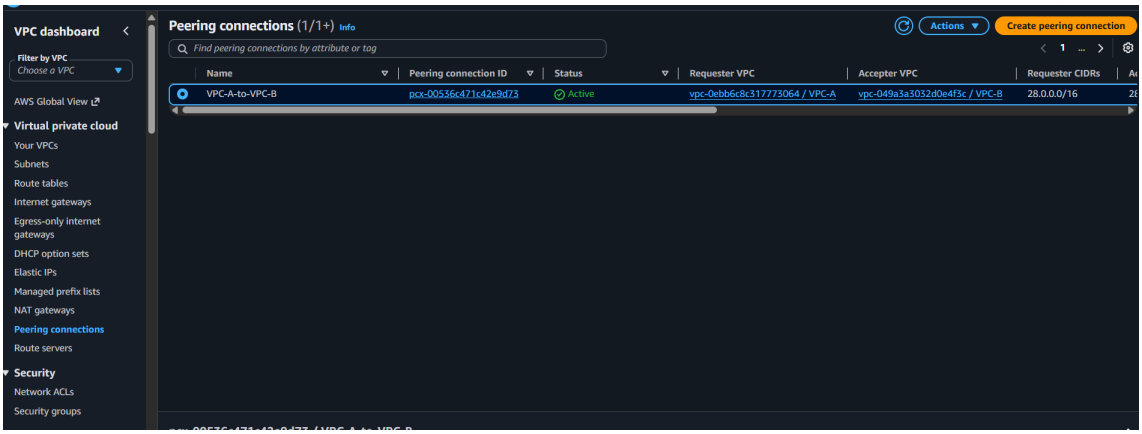


Figure 6. Peering connections list: VPC-A-to-VPC-B, Active, requester CIDR 28.0.0.0/16.

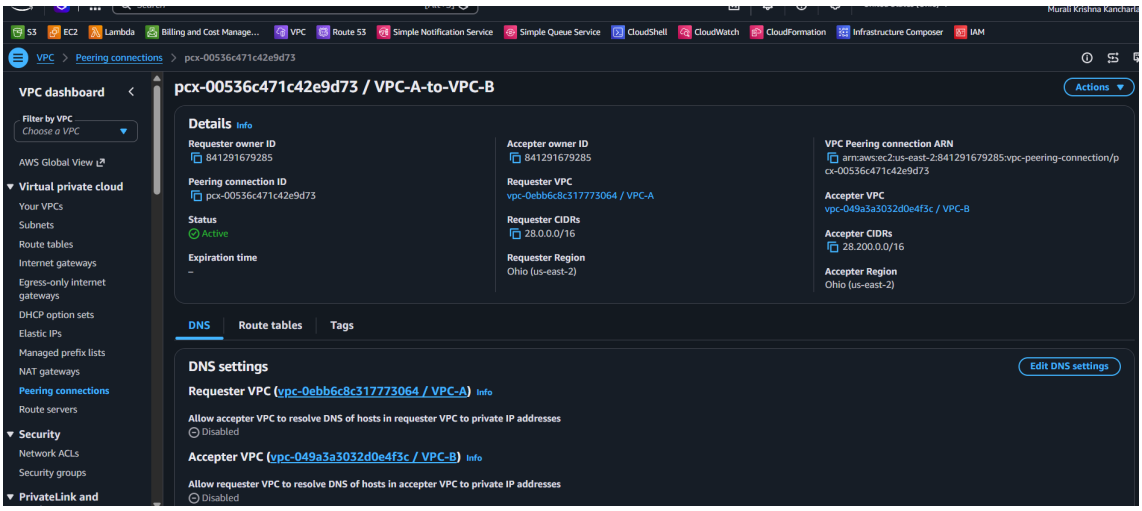


Figure 7. Peering connection pcx-00536c471c42e9d73 / VPC-A-to-VPC-B: Status Active, requester VPC-A (28.0.0.0/16), accepter VPC-B (28.200.0.0/16), both in us-east-2.

06 / SECURITY GROUPS

Inbound rules

Only the traffic needed for SSH and private communication.

Bastion-SG - sg-0656999fdb60ae672 (VPC-A)

Rule ID	Type	Protocol / port	Source
sgr-099b15862a81e87d0	SSH (IPv4)	TCP / 22	0.0.0.0/0

The console shows 1 inbound rule and 1 outbound rule. The outbound rule is not shown, so I cannot list it.

Appserver-SG - sg-05e04db63f87f28db (VPC-A)

Rule ID	Type	Protocol / port	Source
sgr-0603e477d7bde579d	SSH	TCP / 22	sg-0656999fdb60ae67... (Bastion-SG; ID cut off)
sgr-07181d77093287036	All traffic (IPv4)	All / All	28.200.0.0/16

The console shows 2 inbound rules and 1 outbound rule. The outbound rule is not shown.

Remoteapp-SG - sg-0f497072fbef07095 (VPC-B)

Rule ID	Type	Protocol / port	Source
sgr-0da70b3dc78c74034	All traffic (IPv4)	All / All	28.0.0.0/16
sgr-0eee66624945a5e7b	SSH (IPv4)	TCP / 22	28.0.1.10/32

The console shows 2 inbound rules and 1 outbound rule. The outbound rule is not shown. The all-traffic rule allows everything from the whole VPC-A CIDR, which is wider than the workbook's single Echo-request /32 rule. The bastion's private IP in Figure 12 is 28.0.1.240, so the SSH rule for 28.0.1.10/32 does not match it.

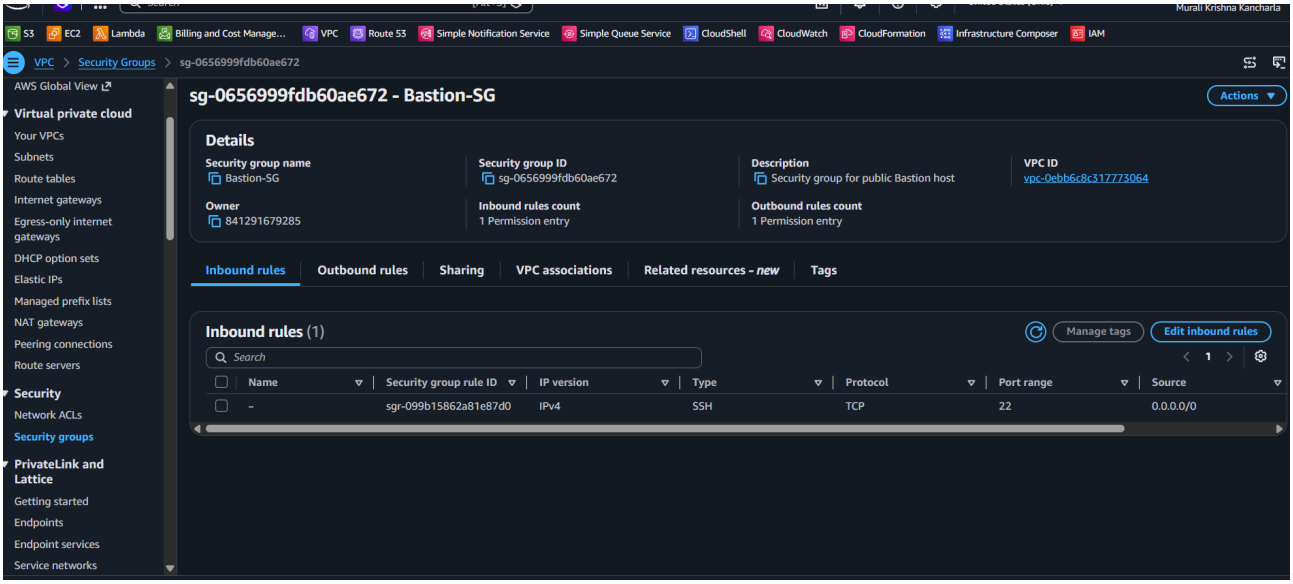


Figure 8. Bastion-SG: one inbound rule, SSH TCP 22 from 0.0.0.0/0.

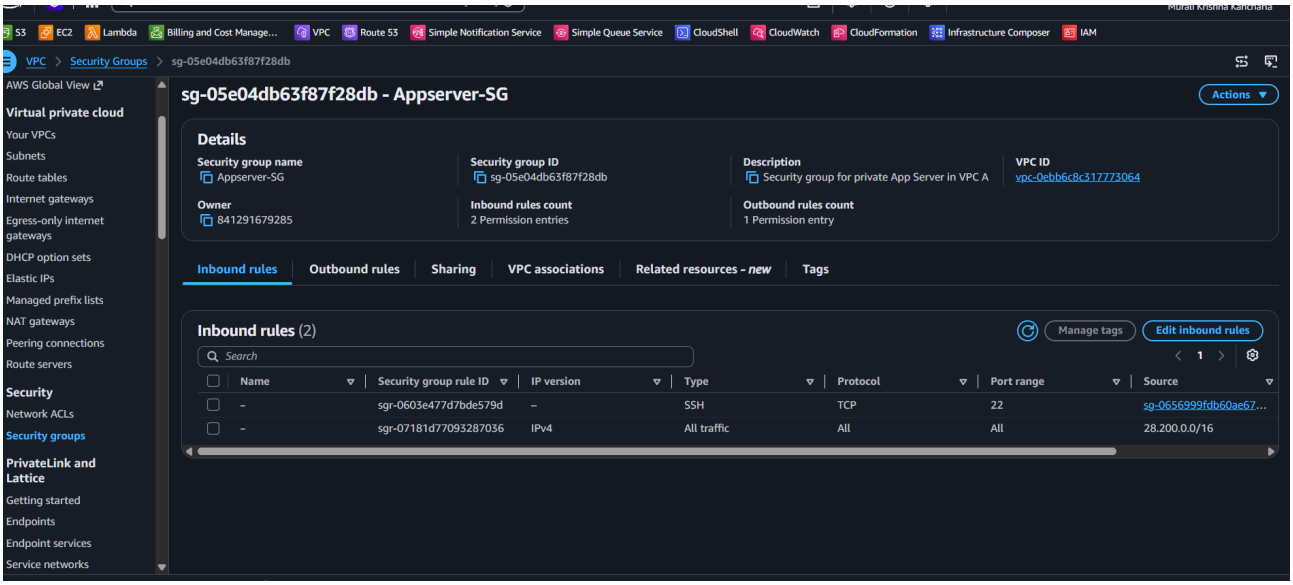


Figure 9. Appserver-SG: SSH (TCP 22) from the Bastion-SG group and All traffic from 28.200.0.0/16.

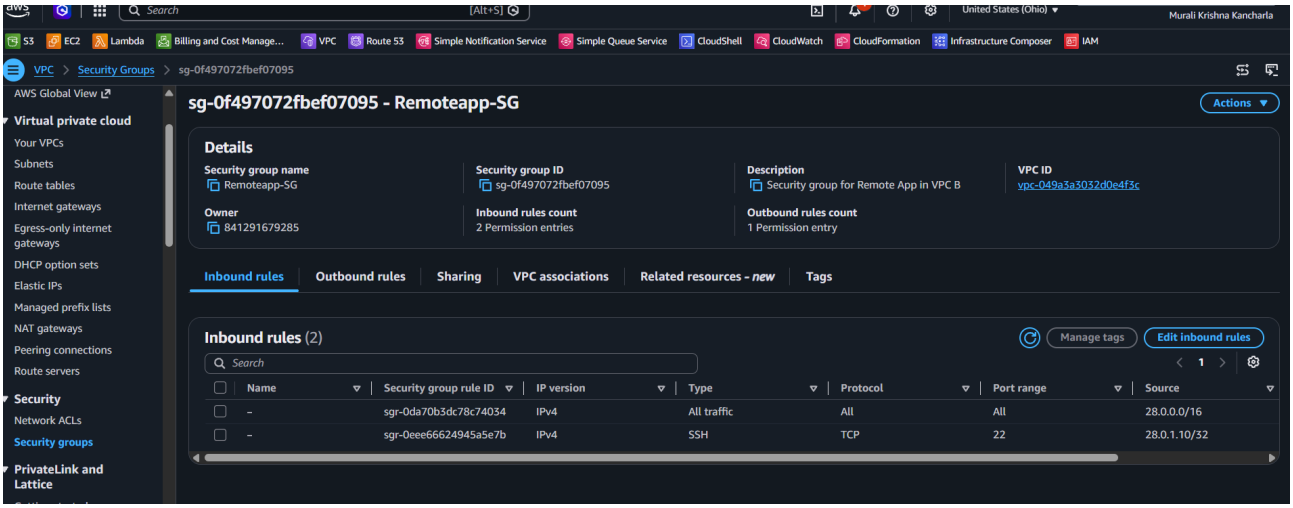


Figure 10. Remoteapp-SG: All traffic from 28.0.0.0/16 and SSH (TCP 22) from 28.0.1.10/32.

07 / COMPUTE

EC2 instances

Three Amazon Linux 2023 instances. Only one has a public IP.

Field	Bastion-vpc-A	App-server-vpc-A	Remote-app-vpc-B
Instance ID	i-0713213471bdb4793	i-0d3a7180a867d28bf	i-0cc4b1af930d4f717
State / status checks	Running / 3 of 3 passed	Running / 3 of 3 passed	Running / 3 of 3 passed
Type / zone	t3.micro / us-east-2a	t3.micro / us-east-2a	t3.micro / us-east-2a
Private IPv4	28.0.1.240	28.0.2.242	28.200.0.10 (inferred)
Public IPv4	3.144.103.155	None (shown as -)	None (shown as -)
Security group	Not visible (Bastion-SG expected)	Not visible (Appserver-SG expected)	Not visible (Remoteapp-SG expected)

The Public IPv4 column for App-server-vpc-A and Remote-app-vpc-B is empty, so those instances have no direct internet address. This matches the workbook design of one bastion and two private application hosts. The instance list does not show which security group each instance uses, so the security group names above are what I intended, not what the console shows.

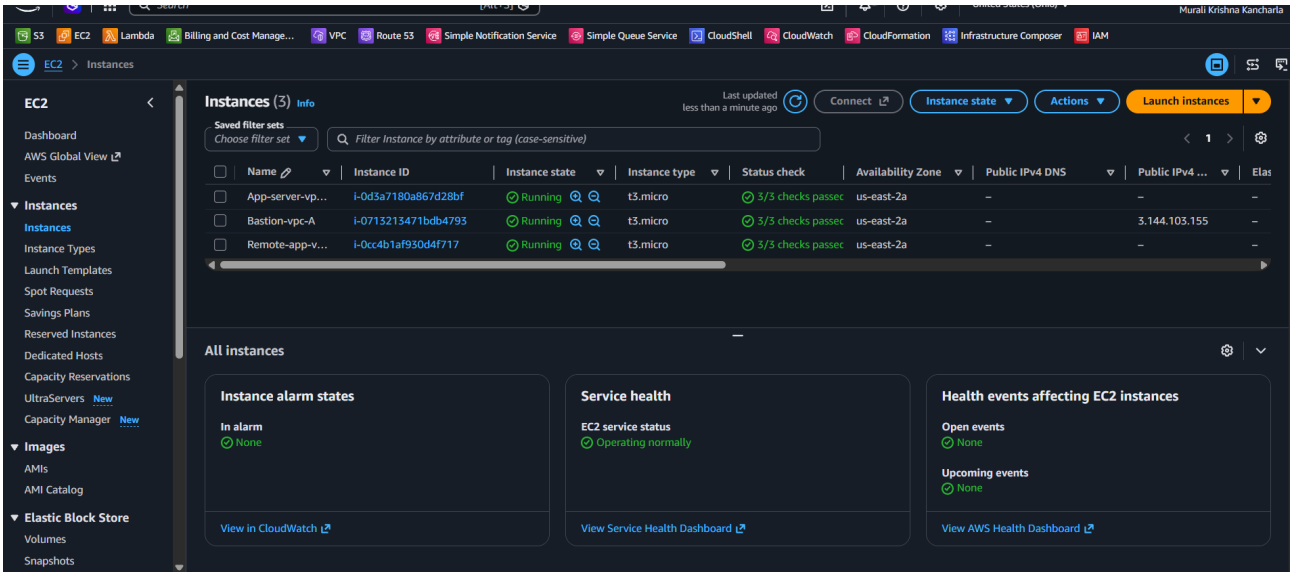


Figure 11. Instances (3): App-server-vp..., Bastion-vpc-A and Remote-app-v..., all t3.micro, Running, 3/3 checks passed, us-east-2a. Only Bastion-vpc-A has a public IPv4 address (3.144.103.155).

08 / VERIFICATION

Connectivity tests

Results from my terminal screenshots.

Test	Where / command	Result	Figure
SSH to bastion	Laptop: ssh -i DEMO.pem ec2-user@3.144.103.155	Login worked (Amazon Linux 2023); prompt shows ip-28-0-1-240	Fig 12
Copy key to bastion	Laptop: scp DEMO.pem to ec2-user@3.144.103.155:~/.ssh/DEMO.pem	100%, 1678 bytes	Fig 13
Private SSH to App-server-vpc-A	Bastion: chmod 400, then ssh -i ~/.ssh/DEMO.pem ec2-user@28.0.2.242	Login worked; prompt shows ip-28-0-2-242	Fig 13
Forward ping across the peering	App-server-vpc-A: ping -c 4 28.200.0.10	**Failed:** 4 sent, 0 received, 100% loss, time 3141 ms	Fig 13
Reverse ping	Remote-app-vpc-B to VPC-A	Not captured	-
Isolation	EC2 instance list	Public IPv4 empty for the app servers	Fig 11
Baseline (before peering)	Workbook test 01: ping before peering and routes	Not captured	-

In Figure 13 I copied my key file to the bastion with scp and then used ssh from the bastion to reach the app server. The workbook uses ProxyJump, which does the same job without leaving a key on the bastion. The bastion login banner in Figure 12 shows an earlier login from 157.50.251.90.

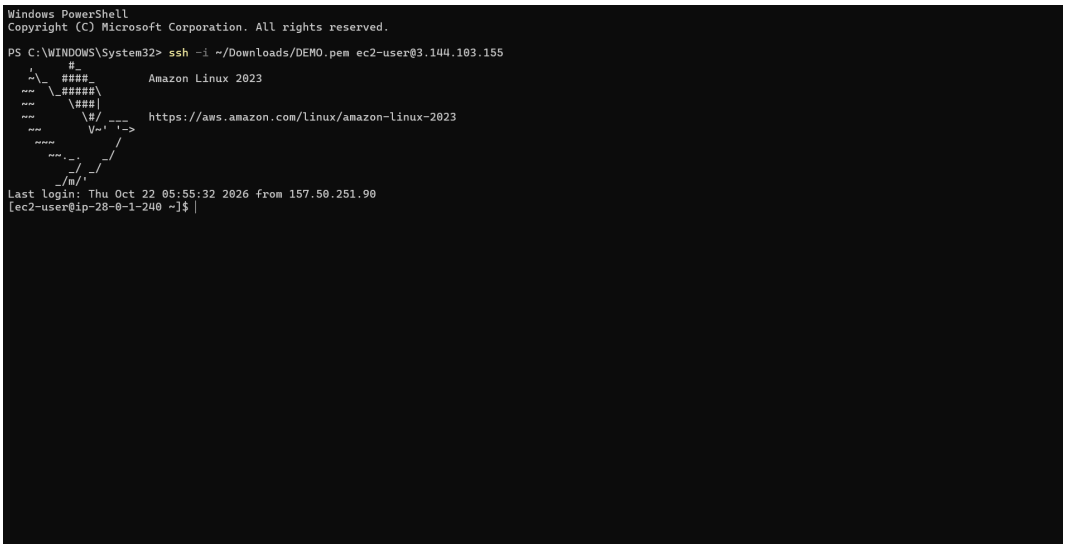


Figure 12. Laptop PowerShell: ssh to the bastion at 3.144.103.155. The prompt shows ip-28-0-1-240, so the bastion's private IP is 28.0.1.240.

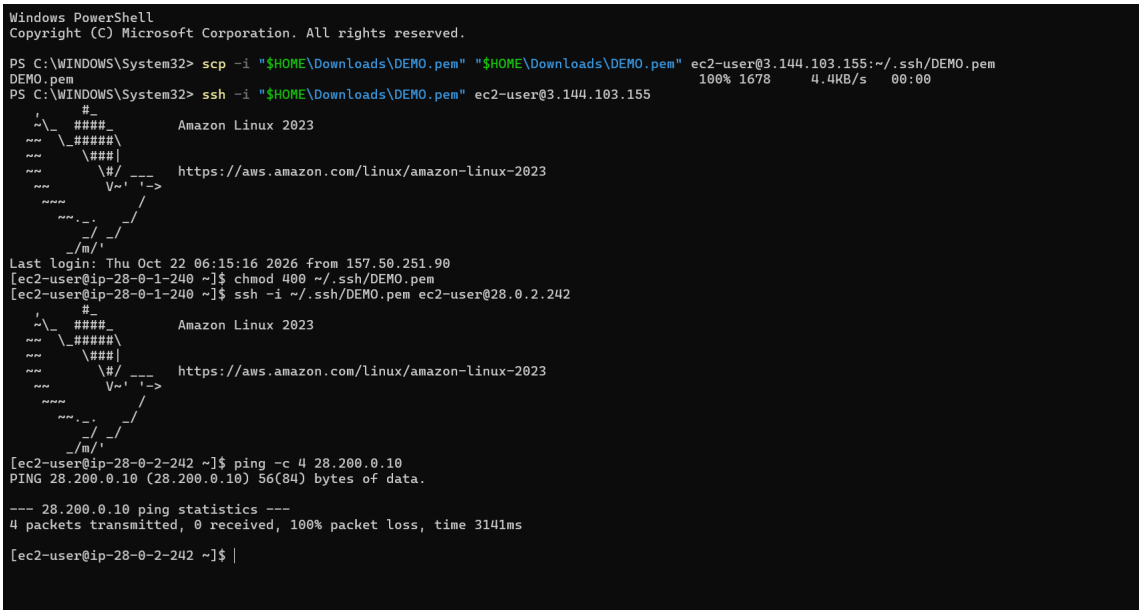


Figure 13. scp of the key to the bastion, ssh from the bastion to 28.0.2.242 (App-server-vpc-A), then ping -c 4 28.200.0.10: 4 sent, 0 received, 100% packet loss.

09 / FAILURE AND RECOVERY

ICMP failure and recovery

What my screenshots show and what they do not.

Step	Command (from App-server-vpc-A)	Result	Figure
Failure	ping -c 4 28.200.0.10	4 sent, 0 received, 100% packet loss, time 3141 ms	Fig 13
Recovery	ping -c 4 28.200.0.10	Not captured	-

What I cannot show
In the workbook, the controlled failure is done by removing the destination's inbound ICMP Echo rule and adding it back. My failure was not a controlled test. I do not have a screenshot of a rule being removed or restored, and Remoteapp-SG in Figure 10 already allows all traffic from 28.0.0.0/16. I also have no screenshot of the route entries in rtb-private-A or rtb-private-B. So the 100% loss is a real result, but I cannot say from my screenshots what caused it, and I have no recovery result. It also cannot count as a before-peering baseline, because the peering connection is already Active.

In general, when ping fails while the peering connection is Active, the problem is usually a route, a security group, a network ACL or the instance firewall, not the peering connection itself. That is general VPC behaviour. My screenshots do not prove it for this case. The things I would check first are that rtb-private-A has a 28.200.0.0/16 route to pcx-00536c471c42e9d73, that rtb-private-B has a 28.0.0.0/16 route back, and that 28.200.0.10 is the real private IP of Remote-app-vpc-B.

10 / TRAFFIC PATHS

Forward path and return path

Why both routes and both security rules are needed.

Forward path: App-server-vpc-A (28.0.2.242) to Remote-app-vpc-B (28.200.0.10)

- The destination 28.200.0.10 should match 28.200.0.0/16 in rtb-private-A, whose target should be pcx-00536c471c42e9d73. I could not confirm this route because the route entries are not in my screenshots.
- The peering connection delivers the packet into VPC-B.
- Remoteapp-SG has to allow it. Its all-traffic rule allows 28.0.0.0/16 (Figure 10), so this part is in place.

Return path: Remote-app-vpc-B back to App-server-vpc-A

- The reply's destination 28.0.2.242 should match 28.0.0.0/16 in rtb-private-B, pointing to the same peering connection. This route is also not visible in my screenshots.
- Security groups are stateful, so replies to allowed traffic are allowed automatically.
- A ping started from VPC-B would depend on Appserver-SG, which allows all traffic from 28.200.0.0/16 (Figure 9). I did not capture that test.

Because the forward ping in Figure 13 failed, I cannot claim that this path works end to end.

Administration path

My laptop connects to 3.144.103.155 on TCP 22 through the internet gateway. From Bastion-vpc-A I then connect to App-server-vpc-A at 28.0.2.242 inside VPC-A. The app servers have no public IPv4 address, so the bastion is the only way in from my laptop. I did not capture an SSH login to Remote-app-vpc-B.

11 / CLEANUP

Cleanup and final resource state

Everything still to be deleted and what the screenshots show.

I have not captured any cleanup screenshots. All screenshots in this report show the lab resources still in place.

Workbook cleanup step	Status
Terminate Bastion-vpc-A, App-server-vpc-A and Remote-app-vpc-B	Not captured. The Instances list (Figure 11) shows all three Running.
Delete lab EBS volumes	Not captured
Delete leftover snapshots	Not captured
Release Elastic IPs	Not captured
Remove unused lab key pairs (DEMO.pem)	Not captured
Delete peering connection and peer routes	Not captured. Peering shows Active (Figure 7).
Delete subnets	Not captured. Three subnets are listed (Figure 3).
Delete security groups	Not captured. Bastion-SG, Appserver-SG and Remoteapp-SG exist (Figures 8 to 10).
Delete custom route tables	Not captured. Figure 4 lists them.
Detach and delete the internet gateway	Not captured. IGW-VPC-A is Attached (Figure 5).
Delete both VPCs	Not captured. Figure 2 lists them.
Check Billing for remaining resources	Not captured

Things to note
The three route tables and the three instances stay billable or in use until I delete them. Instances, the public IPv4 address and any volumes can cost money, so cleanup should be done soon after the lab. Billing data can lag, so a $0.00 figure would not prove that nothing is running.

Final resource state

In my last screenshots for us-east-2, all project resources exist: VPC-A, VPC-B, three subnets, three custom route tables, IGW-VPC-A, the peering connection, three security groups and three running instances.

12 / REFLECTION

What I learned and conclusion

My notes on the project.

Learning note

During this internship, I gained practical experience in AWS networking and learned how different cloud resources work together. Working on the VPC Peering project helped me understand how to connect two separate Virtual Private Clouds and enable communication between instances using their IP addresses inside the VPCs.

I also learned the importance of route tables, security groups, and network rules in controlling communication between resources. Testing connectivity through ping and SSH helped me understand how to troubleshoot network issues and identify configuration mistakes.

Overall, this project gave me a better understanding of AWS networking beyond theoretical concepts. It also taught me the importance of checking configurations carefully, testing connectivity step by step, and cleaning up resources after completing a lab. This experience has increased my confidence in working with cloud technologies and motivated me to learn more about AWS and cloud infrastructure.

Observations from my setup

- Bastion-SG allows SSH from 0.0.0.0/0, which exposes port 22 to the whole internet. Limiting it to my own IP address would be safer.
- Appserver-SG and Remoteapp-SG allow all traffic from the other VPC's whole CIDR. They could be limited to SSH and ICMP from specific addresses.
- The SSH rule for 28.0.1.10/32 in Remoteapp-SG does not match the bastion's private IP (28.0.1.240), and the all-traffic rule already covers SSH from VPC-A, so it is not needed.
- I copied the private key to the bastion with scp. ProxyJump or ssh agent forwarding is better, because the key never sits on a public host.
- I do not have a screenshot of the route entries, so I cannot say why the ping to VPC-B failed. Next time I will capture each change as I make it.
- 28.0.0.0/16 and 28.200.0.0/16 are not private (RFC 1918) ranges. They are fine for a lab, but a real network should use RFC 1918 ranges such as 10.0.0.0/8.

Conclusion

I connected two VPCs in us-east-2 with an Active peering connection. My setup follows the workbook design: a public bastion, a private app server in VPC-A, a remote app server in VPC-B and three custom route tables. SSH worked from my laptop to the bastion and from the bastion to App-server-vpc-A. The ping from App-server-vpc-A to 28.200.0.10 in VPC-B failed with 100% packet loss, and I cannot say what caused it. I did not capture the route entries, a reverse ping, a recovery result, an SSH login to the VPC-B instance or any cleanup. The remaining gaps are listed in Appendix B.

APPENDIX A / SCREENSHOT INDEX

Screenshot index

Original files, figure numbers and the sections they belong to.

Figure	File	Section
Fig 1	Diagram drawn for this report (not a screenshot)	2
Fig 2	Screenshot_2026-10-22_115936.png (Your VPCs)	3
Fig 3	Screenshot_2026-10-22_120032.png (Subnets)	4
Fig 4	Screenshot_2026-10-22_120253.png (Route tables)	4
Fig 5	Screenshot_2026-10-22_120311.png (Internet gateways)	4
Fig 6	Screenshot_2026-10-22_120403.png (Peering connections)	5
Fig 7	Screenshot_2026-10-22_120353.png (Peering connection details)	5
Fig 8	Screenshot_2026-10-22_120645.png (Bastion-SG)	6
Fig 9	Screenshot_2026-10-22_120700.png (Appserver-SG)	6
Fig 10	Screenshot_2026-10-22_120624.png (Remoteapp-SG)	6
Fig 11	Screenshot_2026-10-22_120738.png (Instances)	7
Fig 12	Screenshot_2026-10-22_112819.png (SSH to bastion)	8
Fig 13	Screenshot_2026-10-22_114907.png (scp, SSH to app server, failed ping)	8, 9

I left out these files because they repeat other screenshots: Screenshot_2026-10-22_120324.png (the internet gateways list in Figure 5), and Screenshot_2026-10-22_114850.png and Screenshot_2026-10-22_114900.png (earlier views of Figure 13).

APPENDIX B / COVERAGE

Workbook checklist and remaining gaps

A clear record of what the evidence shows and what it does not show.

Workbook item	Status	Notes
Cover: name, batch, title, region and date	Complete	Included on the cover page.
Architecture diagram and resource inventory	Complete with limitation	Resource IDs and names are listed. Instance placement is inferred from names and IPs, and the VPC-B private IP (28.200.0.10) is inferred.
Subnet CIDRs and route tables	Partly complete	Subnets and table associations are shown. Route entries are not visible.
Active peering and security-group rules	Mostly complete	Peering status and inbound rules for all three groups are captured. Outbound rules are not shown, and the Bastion-SG ID in Appserver-SG is cut off.
Baseline failure before peering	Not captured	No pre-peering test.
Forward/reverse ping and SSH to VPC-B	Partly complete	SSH to the bastion and to the VPC-A app server is shown. The forward ping to VPC-B failed. No reverse ping and no SSH to the VPC-B instance.
Controlled ICMP failure and recovery	Not captured	The failed ping was not a controlled test, and no recovery or rule change was captured.
Cleanup	Not captured	All resources were still in place in the screenshots.
Learning note	Complete	Included in Section 12.

Final submission note

I have kept the evidence gaps visible instead of guessing. The report distinguishes what is read from screenshots from what is inferred, and it lists the remaining gaps. The lab resources have not been cleaned up yet.